R + αR² in FLRW — one-page cheat sheet & notebook bundle
Conventions: signature (−,+,+,+); ; flat FLRW , ; . Unless stated, set .
1) Field equations & definition of
Action (truncation):
Modified Einstein equations (background cosmology uses only component):
with
Do not multiply by in the Friedmann equation.
2) Fully simplified on flat FLRW
Using , , :
Hence the drop-in expression
Friedmann ( ):
3) Scalar–tensor (Jordan) equivalence & the 1/3 rule
, .
M =P −2 8πG ds =2 −dt +2 a (t)d2 x2 H ≡
/aȧ □X = − −Ẍ 3HẊ c = ħ = 1
Qμv
S = d x (R+∫ 4 −g 2 MP 2
αR)+2 S .m
00
G +μv Λg =μv 8πGT +μv (m) Q , Q ≡μv μv αH ,μv
(R)2
H =μv (R)2 2RR −μv g R −2
1 μv
2 2∇ ∇ R+μ v 2g □R.μv
Q00 8πG
Q00
R = 6(2H +2)Ḣ R =00 −3(+Ḣ H)2 □R = − −R̈ 3HṘ
H =00 (R)2 2RR +00 R +2
1 2 6H =Ṙ 108H −2Ḣ 18( ) +Ḣ 2 36H .Ḧ
Q = α[108H − 18( ) + 36H] = 18α(6H − ( ) + 2H).00 2Ḣ Ḣ 2 Ḧ 2Ḣ Ḣ 2 Ḧ
g =00 −1
3H + Λ = 8πGρ +Q .2 m 00
f(R) = R+ αR ⇒2 ϕ ≡ f (R) =′ 1 + 2αR R = (ϕ− 1)/(2α)
Homogeneous gives
which matches the metric-variation after substituting .
Weak-field (Yukawa): linearization reveals a massive scalar ("scalaron") with
The is the fixed spin-0 projector weight in .
4) Einstein-frame map (for inflation or diagnostics)
Starobinsky potential
5) Sanity checks
de Sitter . Radiation era . Matter era (for).
6) Notebook cell bundle (copy/paste in order)
[1] Core utilities & units
import numpy as np

```
# Reduced Planck mass Mpl: choose units. For natural units, set Mpl=1.
# Relation: Mpl^2 = 1/(8*pi*G).
def G_from_Mpl(Mpl):
S = d x [ ϕR−∫ 4 −g 2 MP 2
U (ϕ)]+2 MP 2
J S , U (ϕ) =m J .4α (ϕ−1)2
ϕ(t)
Q = ( U (ϕ) − 3H ),00 (scalar)
ϕ
1 4 MP 2
J ϕ̇
Q00 ϕ = 1 + 2αR
Φ(r) = − [1 + e ], Ψ(r) = − [1 − e ], m = = . r
GM 3 1 −mr
r
GM 3 1 −mr 2
12α MP 2
96πGα
1
1/3 f(R)
ϕ = e , φ =φ/M2/3 P M lnϕ.2 3
P
V (φ) = M m (1 − e ) , m = .E 4 3
P 2 2 − φ/M2/3 P
2 2 12α MP 2
• H = const ⇒ Q =00 0
• a ∝ t ⇒1/2 R = 0 ⇒ Q =00 0
• a ∝ t ⇒2/3 Q =00 −8α/t4 α > 0
return 1.0/(8.0*np.pi*Mpl*Mpl)
def b_coeff(Mpl):
return np.sqrt(2.0/3.0)/Mpl
def m_from_alpha(alpha, Mpl):
# m^2 = Mpl^2/(12*alpha) (natural units)
return Mpl/np.sqrt(12.0*alpha)
[2] Q00 for R+αR² and (optional) pressure
def Q00_R2(H, Hdot, Hddot, alpha):
# 18*alpha*(6 H^2 Hdot - Hdot^2 + 2 H Hddot)
return 18.0*alpha*(6.0*H*H*Hdot - Hdot*Hdot + 2.0*H*Hddot)
# Effective pressure if you use a Q-fluid view (optional):
# p_Q = alpha*(-156 H^2 Hdot - 54 Hdot^2 - 84 H Hddot - 12 H3dot)
[3] Einstein-frame Starobinsky potential & slow-roll
def V_E(varphi, Mpl, alpha=None, m=None):
if m is None:
assert alpha is not None
```

```
m = m_from_alpha(alpha, Mpl)
b = b_coeff(Mpl)
y = np.exp(-b*varphi)
return 0.75*(Mpl*Mpl)*(m*m)*(1.0 - y)**2
def dV_E(varphi, Mpl, alpha=None, m=None):
if m is None:
assert alpha is not None
m = m_from_alpha(alpha, Mpl)
b = b_coeff(Mpl)
y = np.exp(-b*varphi)
A = 0.75*(Mpl*Mpl)*(m*m)
return 2.0*A*b*y*(1.0 - y)
def ddV_E(varphi, Mpl, alpha=None, m=None):
if m is None:
assert alpha is not None
m = m_from_alpha(alpha, Mpl)
b = b_coeff(Mpl)
y = np.exp(-b*varphi)
A = 0.75*(Mpl*Mpl)*(m*m)
return 2.0*A*(b*b)*(-y + 2.0*y*y)
def epsilon_V(varphi, Mpl, alpha=None, m=None):
V = V_E(varphi, Mpl, alpha, m)
dV = dV_E(varphi, Mpl, alpha, m)
return 0.5*(Mpl*Mpl)*(dV/V)**2
def eta_V(varphi, Mpl, alpha=None, m=None):
V = V_E(varphi, Mpl, alpha, m)
d2V = ddV_E(varphi, Mpl, alpha, m)
return (Mpl*Mpl)*(d2V/V)
def ns_r_at(varphi, Mpl, alpha=None, m=None):
eps = epsilon_V(varphi, Mpl, alpha, m)
eta = eta_V(varphi, Mpl, alpha, m)
return 1.0 - 6.0*eps + 2.0*eta, 16.0*eps
def As_amp(varphi, Mpl, alpha=None, m=None):
V = V_E(varphi, Mpl, alpha, m)
eps = epsilon_V(varphi, Mpl, alpha, m)
return V/(24.0*(np.pi**2)*(Mpl**4)*eps)
def varphi_end(Mpl):
b = b_coeff(Mpl)
y_end = 2.0*np.sqrt(3.0) - 3.0
return -np.log(y_end)/b
def N_between(varphi_i, varphi_end_, Mpl):
b = b_coeff(Mpl)
return 0.75*(np.exp(b*varphi_i) - np.exp(b*varphi_end_) - b*(varphi_i -
varphi_end_))
```

```
def varphi_for_N(N_target, Mpl, tol=1e-12, maxit=50):
b = b_coeff(Mpl)
ve = varphi_end(Mpl)
phi = (1.0/b)*np.log(max(4.0*N_target/3.0, 1.1))
for _ in range(maxit):
f = N_between(phi, ve, Mpl) - N_target
if abs(f) < tol:
break
dN = 0.75*b*(np.exp(b*phi) - 1.0)
phi -= f/dN
return phi
[4] Background integrator with mode switching (H / φ / R)
class Background:
"""mode ∈ {"H", "phi", "R"}. Natural units by default (Mpl=1 ⇒ G=1/
(8π))."""
def __init__(self, mode="phi", Mpl=1.0, alpha=1e-2, Lambda=0.0, hbar=1.0,
w=0.0, rho0=0.0):
assert mode in ("H", "phi", "R")
self.mode = mode
self.Mpl = float(Mpl)
self.alpha = float(alpha)
self.Lambda = float(Lambda)
self.hbar = float(hbar)
self.w = float(w)
self.G = G_from_Mpl(self.Mpl)
self.m = m_from_alpha(self.alpha, self.Mpl)
self.state = None
self.t = None
self.rho = rho0
# Jordan scalar potential U_J(φ) = (φ−1)^2/(4α) times Mpl^2/2 factor in
action ⇒ energy density piece below
def U_phi(self, phi):
return (self.Mpl**4)*(phi - 1.0)**2/(16.0*self.alpha)
def dU_phi(self, phi):
return (self.Mpl**4)*(phi - 1.0)/(8.0*self.alpha)
def Hddot_from_Friedmann(self, H, Hdot, rho):
lhs = 3.0*H*H
rhs_no_Hdd = self.Lambda + 8.0*np.pi*self.G*rho +
self.hbar*18.0*self.alpha*(6.0*H*H*Hdot - Hdot*Hdot)
denom = self.hbar*18.0*self.alpha*(2.0*H)
if abs(denom) < 1e-18:
denom = 1e-18 if denom == 0 else np.sign(denom)*1e-18
return (lhs - rhs_no_Hdd)/denom
def H_from_phi_constraint(self, phi, phidot, rho):
```

```
S = 8.0*np.pi*self.G*rho + 0.5*self.U_phi(phi)
A, B, C = 3.0*phi, 3.0*phidot, -S
disc = B*B - 4.0*A*C
disc = 0.0 if disc < 0 else disc
return (-B + np.sqrt(disc))/(2.0*A) # expanding branch
def f_H_mode(self, t, y):
a, H, Hdot, rho = y
Hddot = self.Hddot_from_Friedmann(H, Hdot, rho)
adot = a*H
rhodot = -3.0*H*(1.0 + self.w)*rho
return np.array([adot, Hdot, Hddot, rhodot])
def f_phi_mode(self, t, y):
a, phi, phidot, rho = y
H = self.H_from_phi_constraint(phi, phidot, rho)
T = rho*(-1.0 + 3.0*self.w)
phiddot = (8.0*np.pi*self.G*T - (2.0*self.U_phi(phi) -
phi*self.dU_phi(phi)))/3.0 - 3.0*H*phidot
adot = a*H
rhodot = -3.0*H*(1.0 + self.w)*rho
return np.array([adot, phidot, phiddot, rhodot]) * 0.0 + np.array([adot,
phidot, phiddot, rhodot])
def R_from_phi(self, phi):
return (self.Mpl*self.Mpl)*(phi - 1.0)/(4.0*self.alpha)
@staticmethod
def rk4_step(f, t, y, dt):
k1 = f(t, y)
k2 = f(t + 0.5*dt, y + 0.5*dt*k1)
k3 = f(t + 0.5*dt, y + 0.5*dt*k2)
k4 = f(t + dt, y + dt*k3)
return y + (dt/6.0)*(k1 + 2*k2 + 2*k3 + k4)
def set_ic(self, t0, **kwargs):
self.t = float(t0)
if self.mode == "H":
a0 = kwargs.get("a0", 1.0)
H0 = kwargs["H0"]
Hdot0 = kwargs.get("Hdot0", 0.0)
self.rho = kwargs.get("rho0", self.rho)
self.state = np.array([a0, H0, Hdot0, self.rho], float)
elif self.mode == "phi":
a0 = kwargs.get("a0", 1.0)
phi0 = kwargs["phi0"]
phidot0 = kwargs.get("phidot0", 0.0)
self.rho = kwargs.get("rho0", self.rho)
self.state = np.array([a0, phi0, phidot0, self.rho], float)
```

```
else: # R-mode
a0 = kwargs.get("a0", 1.0)
R0 = kwargs["R0"]
Rdot0 = kwargs.get("Rdot0", 0.0)
self.rho = kwargs.get("rho0", self.rho)
phi0 = 1.0 + 4.0*self.alpha*R0/(self.Mpl*self.Mpl)
phidot0 = 4.0*self.alpha*Rdot0/(self.Mpl*self.Mpl)
self.state = np.array([a0, phi0, phidot0, self.rho], float)
def step(self, dt):
if self.mode == "H":
self.state = self.rk4_step(self.f_H_mode, self.t, self.state, dt)
else:
self.state = self.rk4_step(self.f_phi_mode, self.t, self.state, dt)
self.t += dt
return self.state.copy()
def snapshot(self):
if self.mode == "H":
a, H, Hdot, rho = self.state
out = dict(t=self.t, a=a, H=H, Hdot=Hdot, rho=rho)
out["Hddot"] = self.Hddot_from_Friedmann(H, Hdot, rho)
return out
else:
a, phi, phidot, rho = self.state
H = self.H_from_phi_constraint(phi, phidot, rho)
out = dict(t=self.t, a=a, H=H, phi=phi, phidot=phidot, rho=rho)
out["R"] = self.R_from_phi(phi)
out["Rdot"] = (self.Mpl*self.Mpl)*phidot/(4.0*self.alpha)
return out
[5] Quickstart snippets
# Choose units
Mpl = 1.0
alpha = 1e-2
w = 0.0
rho0 = 0.0
# Einstein-frame diagnostics
a_phi_end = varphi_end(Mpl)
a_phi_60 = varphi_for_N(60.0, Mpl)
ns, r = ns_r_at(a_phi_60, Mpl, alpha=alpha)
As = As_amp(a_phi_60, Mpl, alpha=alpha)
# Background in φ-mode
bg = Background(mode="phi", Mpl=Mpl, alpha=alpha, w=w, rho0=rho0)
bg.set_ic(t0=0.0, a0=1.0, phi0=1.0, phidot0=0.0, rho0=rho0)
traj = []
for n in range(5000):
```

```
bg.step(dt=1e-3)
if n % 50 == 0:
traj.append(bg.snapshot())
# H-only mode (algebraic Hddot)
bgH = Background(mode="H", Mpl=Mpl, alpha=alpha, w=w, rho0=rho0)
bgH.set_ic(t0=0.0, a0=1.0, H0=1e-5, Hdot0=0.0, rho0=rho0)
cur = bgH.snapshot()
```

7) Unit warnings

In SI, has dimensions of . The scalaron Compton wavenumber is in .

In natural units ( ), using (reduced), with in .

• α [length]2 m = 1/ 96πGα

[1/length]

• c = ħ = 1 MP m =2 M /(12α)P

2 α [energy]−2